a.

No, I transformed the data into polar coordinates and a parabolic function and logarithmic spiral functions, and I tried on the third column, but only achieved accuracies of 40.6% to 58.7%. I used the Kernel trick to map data in lower dimensions by measuring similarity between pairs of data points and decomposing the similarity matrix into orthogonal components. This yielded 62.5% accuracy but was still too low to achieve a distinct decision boundary.

b.

1st plot:

Settings: n_iters = 100000

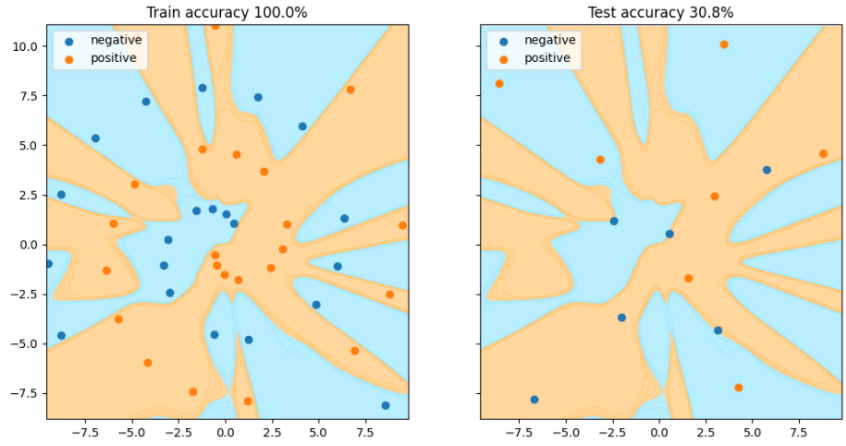
learning_rate = 1

reg = Regularizer(alpha=0, penalty="l2")

layers = [

FullyConnected(2, 32, regularizer=reg),

SigmoidActivation(), FullyConnected(32, 1, regularizer=reg), SigmoidActivation()]



2nd Plot:

Settings: n_iters = 100000

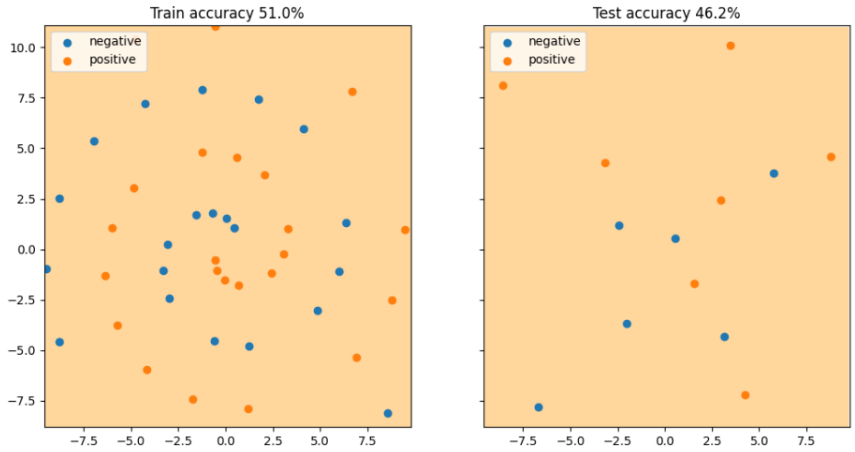
learning_rate = 1

reg = Regularizer(alpha=0.1, penalty="l2")

layers = [

FullyConnected(2, 32, regularizer=reg),

SigmoidActivation(), FullyConnected(32, 1, regularizer=reg), SigmoidActivation()]



3rd Plot:

Settings: n_iters = 100000

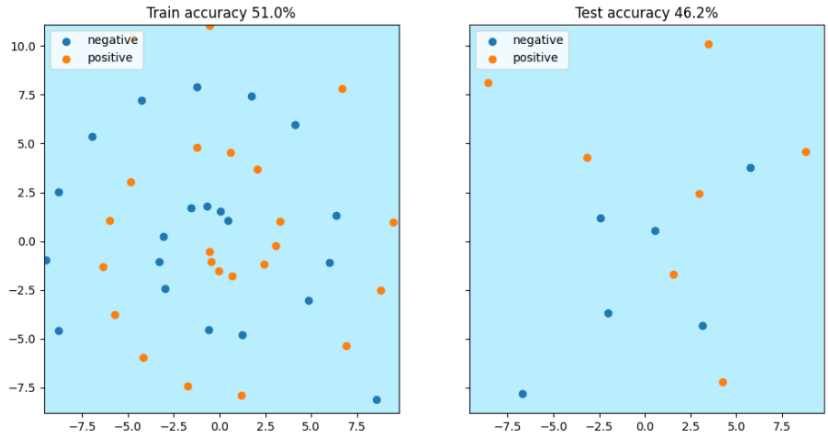
learning_rate = 1

reg = Regularizer(alpha=0.5, penalty="l2")

layers = [

FullyConnected(2, 32, regularizer=reg),

SigmoidActivation(), FullyConnected(32, 1, regularizer=reg), SigmoidActivation()]


Train accuracy 51.0%
negative
positive
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
Test accuracy 46.2%
negative
positive
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5